

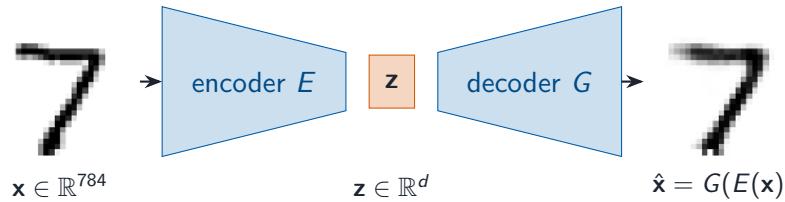
Lecture 6: Generative Models
Variational and Conditional Autoencoders
Generated Spins and the Hamiltonian

Deep Learning in Classical and Quantum Physics

Marcin Płodzień

Atomic Optics Department
Institute of Theoretical Physics
Jagiellonian University in Kraków

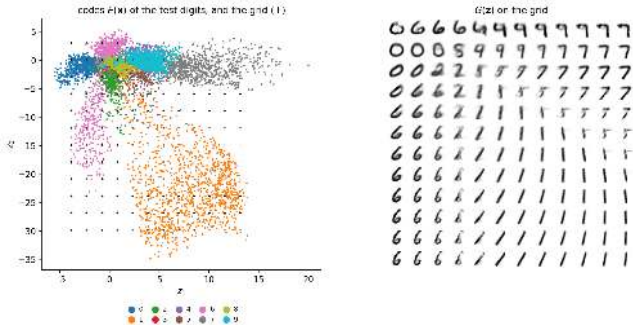
Recap: the Autoencoder and its Latent Space



- trained on images alone, $\mathcal{L} = \frac{1}{n}\sum_i \frac{1}{784}\|\mathbf{x}_i - G(E(\mathbf{x}_i))\|^2$; the bottleneck $d \ll 784$ is a learned, lossy compression (Lecture 5, Lab 017)
- similar images get nearby latent vectors $\mathbf{z}$; the decoder turns *any* $\mathbf{z}$ into an image

Fed with latent vectors that no training image was mapped to, does the decoder produce new digits?

Decoding the Latent Plane of an Autoencoder



Autoencoder with $d = 2$. Left: $\mathbf{z} = E(\mathbf{x})$ for the 10 000 test digits; right: $G(\mathbf{z})$ on a 12×12 grid covering the codes. Coordinates of very different spread, $\mathrm{std}(z_1, z_2) = (3.8, 7.5)$; the 1s fill most of the plane, the other digits crowd together; between clusters: mixtures. **No distribution tells us where, and how often, to sample.**

Random Latent Vectors Give No Digits



Autoencoder with $d = 16$, decoder applied to random $\mathbf{z}$. Top: $\mathbf{z} \sim \mathcal{N}(0, I)$; bottom: $z_k \sim \mathcal{N}(m_k, s_k^2)$, with m_k, s_k the mean and standard deviation of $E(\mathbf{x})_k$ over the training set.

- the codes lie on a thin, curved set in $\mathbb{R}^{16}$; a random point almost never falls on it, even with the right mean and spread per coordinate
- a CNN classifier (Section 3) is confident, $\max_c p(c \mid \mathbf{x}) > 0.9$, for 37% of such images, against 96% of real test digits

To generate we need a latent space with a known distribution.

1

Learning a Probability Distribution



JAGIELLONIAN UNIVERSITY
IN KRAKOW

Generative Models

The data $\mathbf{x}_1, \ldots, \mathbf{x}_n$ are **samples** from an unknown probability distribution $p(\mathbf{x})$: handwritten digits, or spin configurations at temperature T, $p(\mathbf{s} \mid T) = e^{-H(\mathbf{s})/T}/Z$.

A **generative model**

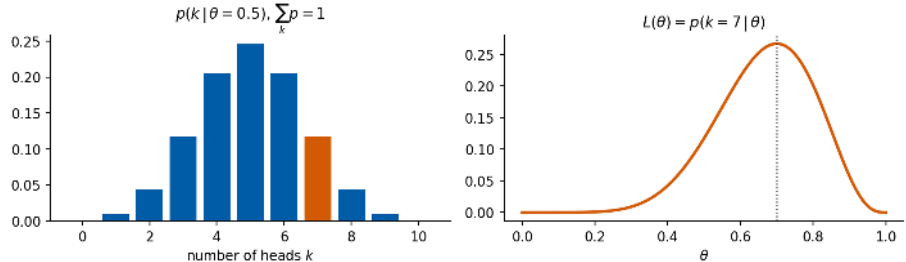
- learns an approximation $p_\theta(\mathbf{x}) \approx p(\mathbf{x})$ from the samples (θ: its parameters)
- draws **new samples** from it: instances that are not in the data set, but could have been

In science: new molecules and materials, ensembles of weather forecasts, fast surrogates of expensive simulations, and Monte Carlo samplers for lattice models.

Two steps: fit a distribution to data (maximum likelihood), and make it easy to sample (latent variables).

Probability and Likelihood

A model $p(x \mid \theta)$ is read in two ways: **probability** (θ fixed, x varies, $\sum_x p = 1$) and **likelihood** (data fixed, θ varies, $L(\theta) = p(x_{\text{obs}} \mid \theta)$, not a distribution over θ).



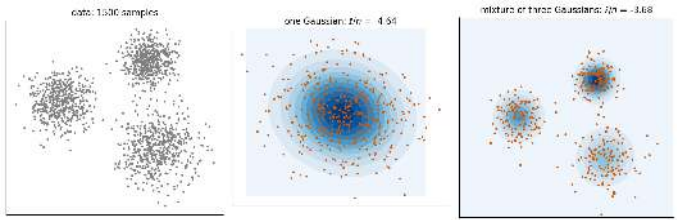
A coin tossed 10 times, $p(k \mid \theta) = \binom{10}{k}\theta^k(1-\theta)^{10-k}$. Left: $p(7 \mid 0.5) = 0.117$. Right: $k = 7$ observed, $\arg\max_\theta L = 0.7$, $\int_0^1 L\, d\theta = 1/11$.

Maximum Likelihood

Independent samples $\mathbf{x}_1, \ldots, \mathbf{x}_n$, a model $p_\theta(\mathbf{x})$:

$$\ell(\theta) = \log \prod_{i=1}^{n} p_\theta(\mathbf{x}_i) = \sum_{i=1}^{n} \log p_\theta(\mathbf{x}_i), \qquad \theta^* = \arg\max_\theta \ell(\theta)$$

$\frac{1}{n}\ell \to -H(p) - \mathrm{KL}(p \,\|\, p_\theta)$ for $n \to \infty$: maximum likelihood minimises $\mathrm{KL}(p \,\|\, p_\theta)$.



$p_\theta = \sum_{k=1}^{K} \pi_k \mathcal{N}(\mathbf{x}; \boldsymbol{\mu}_k, \Sigma_k)$, $\pi = \mathrm{softmax}(\mathbf{a})$, $\Sigma_k = L_k L_k^\top$; gradient ascent on ℓ; orange: 400 new samples. $\ell/n = -4.64$ ($K = 1$), -3.68 ($K = 3$). For images: a neural network.

2

Variational Autoencoders

JAGIELLONIAN UNIVERSITY
IN KRAKOW

A Latent-Variable Model

A fixed **prior**, and a network giving pixel probabilities $\hat{\mathbf{x}} = \sigma(G_\theta(\mathbf{z}))$:

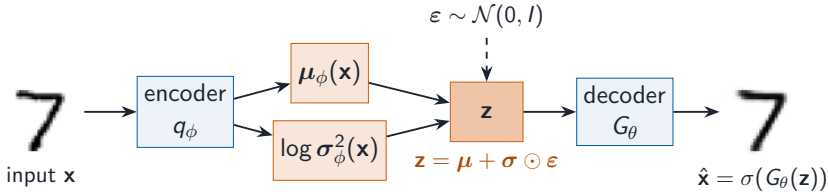
$$\mathbf{z} \sim \mathcal{N}(0, I_d), \qquad p_\theta(\mathbf{x} \mid \mathbf{z}) = \prod_{j=1}^{784} \hat{x}_j^{x_j} (1 - \hat{x}_j)^{1 - x_j}, \qquad p_\theta(\mathbf{x}) = \int p_\theta(\mathbf{x} \mid \mathbf{z})\, p(\mathbf{z})\, d\mathbf{z}$$

$-\log p_\theta(\mathbf{x}|\mathbf{z}) = -\sum_j [x_j \log \hat{x}_j + (1 - x_j) \log(1 - \hat{x}_j)] = \mathrm{BCE}(\mathbf{x}, \hat{\mathbf{x}})$ (binary cross-entropy).

- **generating is easy:** draw $\mathbf{z} \sim \mathcal{N}(0, I)$, decode
- **maximum likelihood is hard:** the integral has no closed form, and sampling it from the prior fails: almost no $\mathbf{z}$ produces a given $\mathbf{x}$. Trained $d = 16$ VAE, $K = 1000$ samples, test average: $\log p_\theta(\mathbf{x}) \approx -154.8$ (prior), -93.3 (importance sampling with the encoder)
- the fix: an **encoder** $q_\phi(\mathbf{z} \mid \mathbf{x})$ proposes the $\mathbf{z}$ that could have produced $\mathbf{x}$

D. P. Kingma, M. Welling, ICLR 2014, arXiv:1312.6114; D. J. Rezende, S. Mohamed, D. Wierstra, ICML 2014, PMLR 32, 1278

Variational Autoencoder: the Architecture



- encoder $q_\phi(\mathbf{z}|\mathbf{x}) = \mathcal{N}(\boldsymbol{\mu}_\phi(\mathbf{x}), \operatorname{diag} \boldsymbol{\sigma}^2_\phi(\mathbf{x}))$: a Gaussian *cloud* instead of a point; outputs $\boldsymbol{\mu}$ and $\log \boldsymbol{\sigma}^2$
- training decodes a random point of the cloud, so every $\mathbf{z}$ near $\boldsymbol{\mu}(\mathbf{x})$ must decode to something like $\mathbf{x}$
- generation skips the encoder: $\mathbf{z} \sim \mathcal{N}(0, I)$, $\hat{\mathbf{x}} = \sigma(G_\theta(\mathbf{z}))$
- MNIST: $784 \to 400 \to 200 \to (\boldsymbol{\mu}, \log \boldsymbol{\sigma}^2)$ and $d \to 200 \to 400 \to 784$, ReLU, $d = 2, 16$

The Evidence Lower Bound

Goal: maximise $\log p_\theta(\mathbf{x}) = \log \int p_\theta(\mathbf{x}|\mathbf{z})\, p(\mathbf{z})\, d\mathbf{z}$, an integral we cannot compute.

1. Bayes' rule, for every $\mathbf{z}$: $\log p_\theta(\mathbf{x}) = \log p_\theta(\mathbf{x}|\mathbf{z}) + \log p(\mathbf{z}) - \log p_\theta(\mathbf{z}|\mathbf{x})$.

2. Insert the encoder $q_\phi(\mathbf{z}|\mathbf{x})$ and average over it, $\mathbb{E}_q[f] = \int q(\mathbf{z}) f(\mathbf{z})\, d\mathbf{z}$; $\mathrm{KL}(q\|p) = \mathbb{E}_q \log(q/p)$:

$$\log p_\theta(\mathbf{x}) = \underbrace{\mathbb{E}_{q_\phi(\mathbf{z}|\mathbf{x})}\big[\log p_\theta(\mathbf{x}\,|\,\mathbf{z})\big] - \mathrm{KL}\big(q_\phi(\mathbf{z}|\mathbf{x}) \,\|\, p(\mathbf{z})\big)}_{\mathrm{ELBO}(\theta,\phi;\mathbf{x})} + \underbrace{\mathrm{KL}\big(q_\phi(\mathbf{z}|\mathbf{x}) \,\|\, p_\theta(\mathbf{z}|\mathbf{x})\big)}_{\geq 0}$$

3. $\mathrm{KL} \geq 0$ (Jensen: $-\mathrm{KL} = \mathbb{E}_q \log \frac{p}{q} \leq \log \mathbb{E}_q \frac{p}{q} = \log 1 = 0$) $\Rightarrow \log p_\theta(\mathbf{x}) \geq \mathrm{ELBO}$: the **E**vidence **L**ower **BO**und, tight when q_ϕ is the true posterior.
Maximising the ELBO raises $\log p_\theta(\mathbf{x})$ and pulls q_ϕ towards $p_\theta(\mathbf{z}|\mathbf{x})$. All of it is computable; the KL of two Gaussians:

$$\mathrm{KL}\big(\mathcal{N}(\boldsymbol{\mu}, \boldsymbol{\sigma}^2) \,\|\, \mathcal{N}(0, I)\big) = \frac{1}{2} \sum_{k=1}^{d} \left(\mu_k^2 + \sigma_k^2 - \ln \sigma_k^2 - 1\right)$$

The VAE Loss, and a Variational Free Energy

Loss per image, $-\mathrm{ELBO}$, with one sample $\mathbf{z} \sim q_\phi(\mathbf{z}|\mathbf{x})$:

$$\mathcal{L}(\mathbf{x}) = \underbrace{\mathrm{BCE}(\mathbf{x}, \sigma(G_\theta(\mathbf{z})))}_{\text{reconstruction}} + \underbrace{\frac{1}{2}\sum_k \left(\mu_k^2 + \sigma_k^2 - \ln \sigma_k^2 - 1\right)}_{\text{stay close to the prior}}$$

- reconstruction: samples of each cloud decode back to $\mathbf{x}$; alone, it shrinks the clouds to points (an autoencoder)
- KL: pulls every cloud to $\mathcal{N}(0, I)$; the clouds overlap and fill the prior, so a fresh $\mathbf{z} \sim \mathcal{N}(0, I)$ decodes to a valid image

Free energy. States $\mathbf{z}$, energy $E(\mathbf{z})$, temperature T: Boltzmann $e^{-E/T}/Z$, $F = -T \log Z$; for a trial distribution q, $F[q] = \langle E \rangle_q - TS[q]$, $S[q] = -\mathbb{E}_q \log q$. With $E(\mathbf{z}) = -\log[p_\theta(\mathbf{x}|\mathbf{z})p(\mathbf{z})]$, $T = 1$: $e^{-E}/Z = p_\theta(\mathbf{z}|\mathbf{x})$, $Z = p_\theta(\mathbf{x})$, and

$$F[q] = F + \mathrm{KL}(q \,\|\, e^{-E}/Z) \geq F \quad \text{(Gibbs–Bogoliubov)}, \qquad F[q] = -\mathrm{ELBO}$$

The Reparametrisation Trick

$\mathbb{E}_q$ is estimated with one sample of $\mathbf{z}$ per image. But "draw $\mathbf{z}$ from $\mathcal{N}(\boldsymbol{\mu}, \boldsymbol{\sigma}^2)$" has no derivative with respect to $\boldsymbol{\mu}$ and $\boldsymbol{\sigma}$.

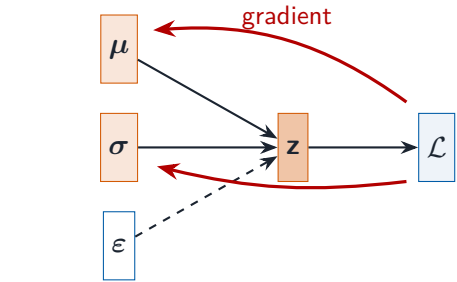
Move the randomness out of the network:

$$\mathbf{z} = \boldsymbol{\mu} + \boldsymbol{\sigma} \odot \boldsymbol{\varepsilon}, \qquad \boldsymbol{\varepsilon} \sim \mathcal{N}(0, I), \qquad \boldsymbol{\sigma} = e^{\frac{1}{2}\log \boldsymbol{\sigma}^2}$$

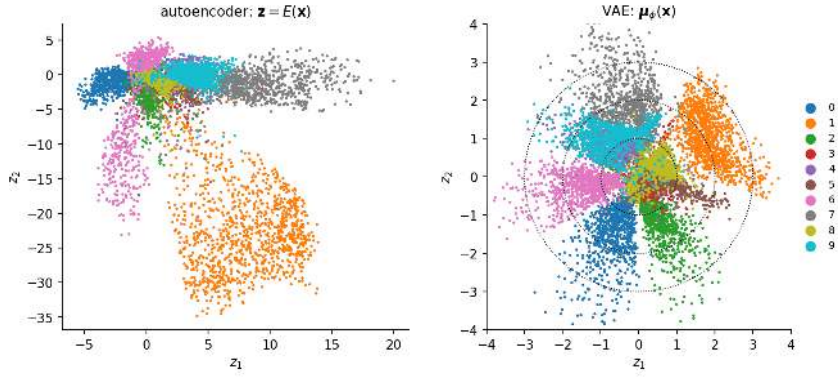
$$\frac{\partial z_k}{\partial \mu_k} = 1, \qquad \frac{\partial z_k}{\partial \sigma_k} = \varepsilon_k$$

$\boldsymbol{\varepsilon}$ does not depend on the parameters; backpropagation runs from the loss through decoder, $\mathbf{z}$ and encoder.

In JAX, $\boldsymbol{\varepsilon}$ comes from an explicit key: `eps = jax.random.normal(key, mu.shape)`.

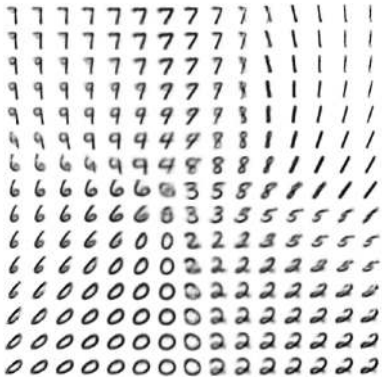


The Latent Plane: Autoencoder and VAE

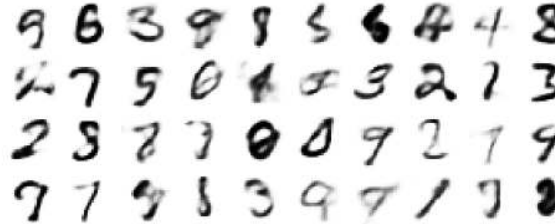


$d = 2$, the 10 000 test digits. Left: $\mathbf{z} = E(\mathbf{x})$; right: $\boldsymbol{\mu}_\phi(\mathbf{x})$, dotted: $|\mathbf{z}| = 1, 2, 3$. Standard deviation of the coordinates: 3.8, 7.5 (AE), 1.03, 1.09 (VAE). The VAE codes fill the region where $\mathcal{N}(0, I)$ puts its probability: **we know where to sample.**

New Digits from $\mathbf{z} \sim \mathcal{N}(0, I)$



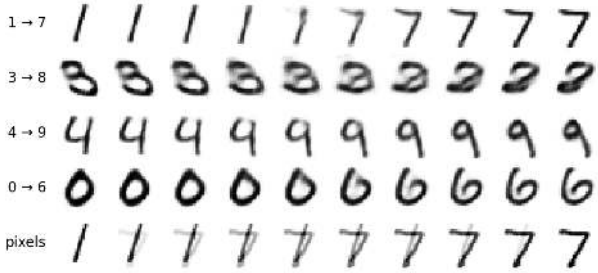
$d = 2$: $\sigma(G_\theta(\mathbf{z}))$, $\mathbf{z}$ on a grid in $[-2.5, 2.5]^2$



$d = 16$: 40 samples, $\mathbf{z} \sim \mathcal{N}(0, I)$

- every grid point decodes to a digit
- not copies: $\langle \min_i \|\hat{\mathbf{x}} - \mathbf{x}_i\| \rangle$ over the training set, 4.48 (generated), 4.19 (real test)
- blurry: we show the pixel probabilities $\hat{x}_j$

Interpolation in the Latent Space

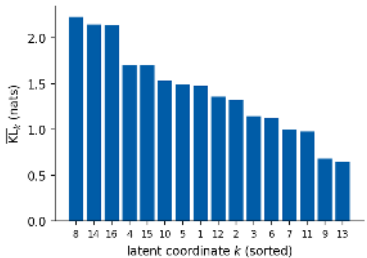


$\mathbf{z}(t) = (1-t)\,\boldsymbol{\mu}_\phi(\mathbf{x}_a) + t\,\boldsymbol{\mu}_\phi(\mathbf{x}_b)$, $t \in [0,1]$, decoded by the $d = 16$ VAE; last row: $(1-t)\,\mathbf{x}_a + t\,\mathbf{x}_b$.

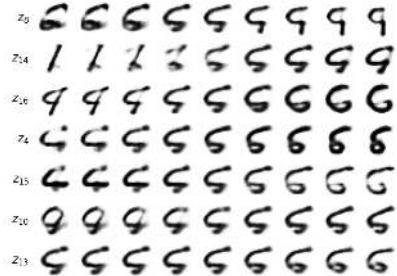
- latent line: plausible digits throughout; the 1 bends into a 7, the 4 closes into a 9
- pixel line: two superposed digits

Straight lines in the latent space stay among the data.

Meaning of the Latent Coordinates



$$\overline{\mathrm{KL}_k} = \left\langle \tfrac{1}{2}(\mu_k^2 + \sigma_k^2 - \ln \sigma_k^2 - 1) \right\rangle_{\mathbf{x}}$$



$\boldsymbol{\mu}_\phi(\mathbf{x})$ of one test 5, one z_k from -3 to 3; top 6 by $\overline{\mathrm{KL}_k}$, and the smallest

- all 16 coordinates are used, unequally: $\overline{\mathrm{KL}_k}$ from 0.65 to 2.23 nats (≈ 0: ignored)
- one coordinate changes slant, width, thickness, and often the digit itself

Limits of Interpreting the Latent Space

We can read

- neighbourhoods: nearby $\mathbf{z}$, similar images
- straight paths: continuous deformations
- usage of each coordinate: $\overline{\mathrm{KL}}_k$
- where the data are: $\mathcal{N}(0, I)$

We cannot read

- the meaning of one axis: $\mathcal{N}(0, I)$ and the KL are unchanged by a rotation $\mathbf{z} \to R\mathbf{z}$, and the decoder absorbs it, $G'(\mathbf{z}) = G(R^\top \mathbf{z})$
- one property per axis ("disentanglement"): not guaranteed without labels or further assumptions

Telling the model what to draw, with a label: the conditional VAE.

F. Locatello et al., *Challenging Common Assumptions in the Unsupervised Learning of Disentangled Representations*, ICML 2019, PMLR 97, 4114

3

Judging Synthetic Data



JAGIELLONIAN UNIVERSITY
IN KRAKÓW

A Classifier as a Referee

Pictures are judged by eye. To *measure* what a generator produces, use a referee trained on the **real** data, with known accuracy on real test data, and apply it unchanged to synthetic images.

- CNN: 3×3 convolutions with 16 and 32 channels, ReLU, each followed by 2×2 max-pooling ($28 \to 14 \to 7$); dense 128; 10 outputs $f_c(\mathbf{x})$
- trained with the cross-entropy, the negative log-likelihood of the labels (Adam, 4 epochs):

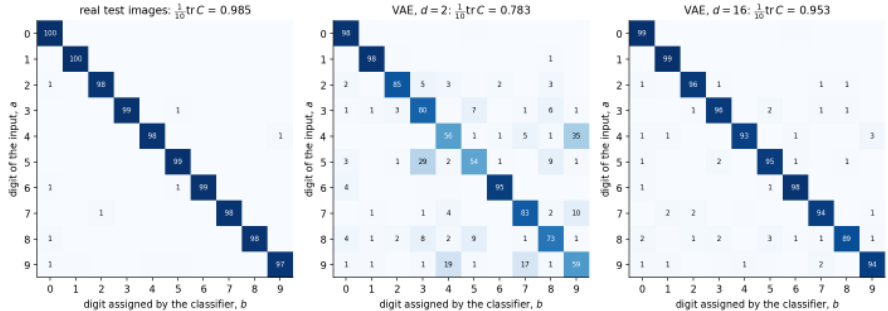
$$\mathcal{L} = -\frac{1}{n}\sum_i \log p(y_i \,|\, \mathbf{x}_i), \qquad p(c \,|\, \mathbf{x}) = \frac{e^{f_c(\mathbf{x})}}{\sum_{c'} e^{f_{c'}(\mathbf{x})}}$$

test accuracy on 10 000 real digits

0.9854

Every test of synthetic data is first run on real data: the baseline.

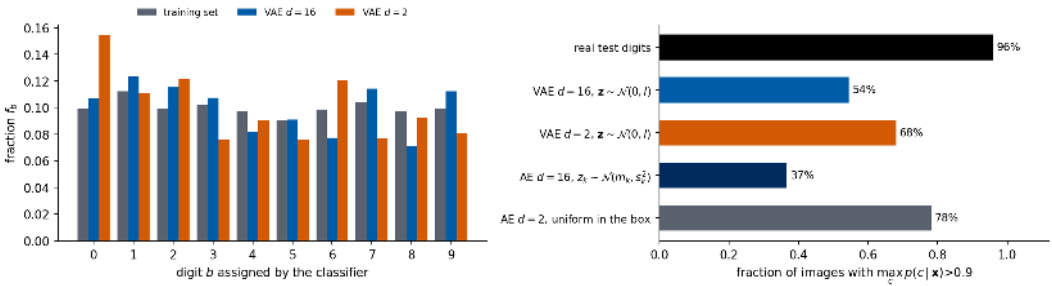
Confusion Matrices of VAE Reconstructions



$C_{ab} = \#\{\text{inputs of digit } a \text{ assigned to } b\}/\#\{\text{inputs of digit } a\}$; row a: digit of the real test input $\mathbf{x}$; column b: classifier label of $\sigma(G_\theta(\mathbf{z}))$, $\mathbf{z} \sim q_\phi(\mathbf{z}\,|\,\mathbf{x})$; in %.

- $d = 16$: $\frac{1}{10}\mathrm{tr}\,C = 0.953$ against 0.985 for real images: the digit survives the latent space
- $d = 2$: look-alike digits share a region of the plane; worst: $4 \to 9$: 35%

Digit Distribution of the Generated Samples



Left: $f_b = \#\{\text{samples assigned to } b\}/N$, $N = 10\,000$, $\mathbf{z} \sim \mathcal{N}(0, I)$. Right: fraction with $\max_c p(c \mid \mathbf{x}) > 0.9$.

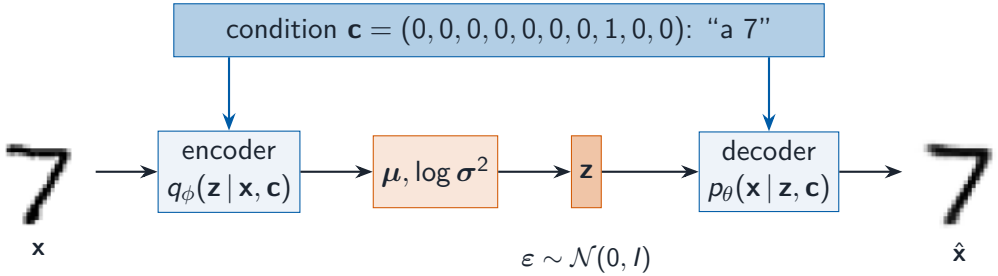
- $d = 16$: every digit, near its training frequency; $d = 2$: too many 0s, too few 3s, 5s, 7s
- many VAE samples are blurry or between two digits; the $d = 2$ AE decodes digit-like images in most of its box, but a uniform box is no model of the data

4

Conditional Generation

JAGIELLONIAN UNIVERSITY
IN KRAKOW

Conditional VAE: the Architecture



- $\mathbf{c}$: a label known for every training image, here the **one-hot** digit, $c_j = \delta_{j,y}$; concatenated to the inputs of both networks: $[\mathbf{x}, \mathbf{c}] \in \mathbb{R}^{794}$ and $[\mathbf{z}, \mathbf{c}] \in \mathbb{R}^{d+10}$
- prior $p(\mathbf{z}) = \mathcal{N}(0, I)$ for every $\mathbf{c}$; loss: BCE + KL as before; one-hot, since a single number y would put 3 "between" 2 and 4

K. Sohn, H. Lee, X. Yan, *Learning Structured Output Representation using Deep Conditional Generative Models*, NeurIPS 2015

The Conditioned ELBO and Generation on Demand

The same derivation with $\mathbf{c}$ carried along gives a lower bound on the conditional likelihood:

$$\log p_\theta(\mathbf{x} \mid \mathbf{c}) \;\geq\; \mathbb{E}_{q_\phi(\mathbf{z}\mid\mathbf{x},\mathbf{c})}\big[\log p_\theta(\mathbf{x} \mid \mathbf{z}, \mathbf{c})\big] - \mathrm{KL}\big(q_\phi(\mathbf{z} \mid \mathbf{x}, \mathbf{c}) \,\big\|\, \mathcal{N}(0, I)\big)$$

Training

pairs $(\mathbf{x}_i, \mathbf{c}_i)$; $\mathbf{z} = \boldsymbol{\mu}_\phi(\mathbf{x}, \mathbf{c}) + \boldsymbol{\sigma}_\phi(\mathbf{x}, \mathbf{c}) \odot \varepsilon$; loss $\mathrm{BCE}(\mathbf{x}, \sigma(G_\theta(\mathbf{z}, \mathbf{c}))) + \mathrm{KL}$

Generation

choose $\mathbf{c}$, draw $\mathbf{z} \sim \mathcal{N}(0, I)$, compute $\sigma(G_\theta(\mathbf{z}, \mathbf{c}))$; no encoder needed

- $\mathbf{c}$ already tells the decoder *which* digit; the KL term penalises every nat stored in $\mathbf{z}$, so $\mathbf{z}$ keeps only what $\mathbf{c}$ does not say: the **style** (slant, width, size)
- $\mathbf{c}$ can be any vector: a one-hot class, a temperature T, a point (g, T) of a phase diagram; MNIST: $d = 8$

Digits on Demand



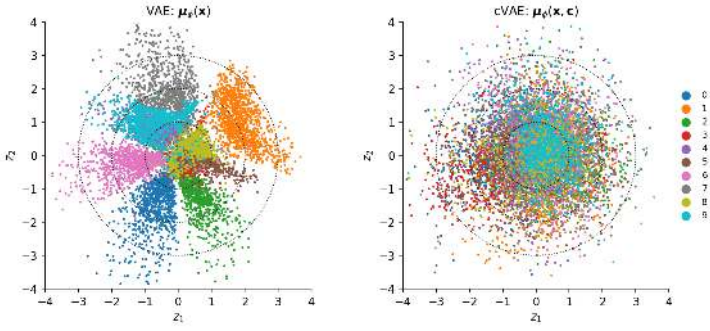
$\sigma(G_\theta(\mathbf{z}, \mathbf{c}))$; row: $\mathbf{c}$ = digit 0 to 9; column: one $\mathbf{z} \sim \mathcal{N}(0, I)$, the same in every row.

Along a column the digit changes and the style stays: $\mathbf{z}$ carries the style, $\mathbf{c}$ the identity.



Style transfer: a real test digit $\mathbf{x}$ (left), $\mathbf{z} = \boldsymbol{\mu}_\phi(\mathbf{x}, \mathbf{c}_y)$, decoded with $\mathbf{c} = 0, \ldots, 9$.

The Latent Plane of a cVAE Holds Only the Style

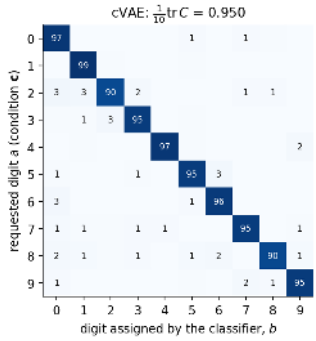


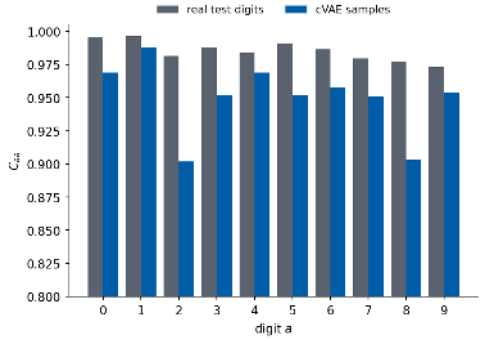


cVAE, $\mathbf{c} = 7$, $\mathbf{z}$ on a grid over $[-2, 2]^2$

$d = 2$; test digits at $\boldsymbol{\mu}_\phi(\mathbf{x})$ (VAE) and $\boldsymbol{\mu}_\phi(\mathbf{x}, \mathbf{c})$ (cVAE). Digit predicted from the nearest class mean in the latent plane: 0.73 (VAE), 0.18 (cVAE), chance 0.1. In the cVAE all digits share one disc: $\mathbf{z}$ is a map of style.

The Requested Digit Is Generated





1000 samples per requested digit, $\mathbf{z} \sim \mathcal{N}(0, I)$.
$C_{ab} = \#\{\text{samples generated with } \mathbf{c} = a \text{ assigned to } b\}/1000$; row a: requested digit; column b: classifier label; in %. Right: the diagonal C_{aa}, next to the classifier on real test digits. Mean C_{aa}: 0.950; largest confusion $2 \to 0$: 3%.

From Digits to Spins

The protocol of this part, ready for physics:

1. a generator conditioned on what we want: $\mathbf{c}$ = the digit $\longrightarrow$ $\mathbf{c} = T$ for the 2D Ising model, $\mathbf{c} = (g, T)$ for the J_1–J_2 model
2. referees trained on **real** data, checked on real test data first $\longrightarrow$ networks trained on Monte Carlo configurations
3. the same tests on synthetic samples

Physics has a stricter referee

the true distribution is known, $p(\mathbf{s} \mid T) = e^{-H(\mathbf{s})/T}/Z$: energy, magnetisation, fluctuations and correlations of generated configurations can be compared with Monte Carlo at every T

The question

do cVAE samples obey the same physics as the Monte Carlo samples they were trained on, including near the phase transition?

5

Generated Spins and the Hamiltonian



JAGIELLONIAN UNIVERSITY
IN KRAKOW

The Hamiltonian Fixes the Distribution

spins $s_i = \pm 1$ on an $L \times L$ lattice, periodic boundaries, $N = L^2$, $k_B = J = J_1 = 1$:

$$H_{\text{Ising}} = -\sum_{\langle ij \rangle} s_i s_j, \qquad H_{J_1 J_2} = -\sum_{\langle ij \rangle} s_i s_j + g \sum_{\langle\langle ij \rangle\rangle} s_i s_j, \qquad p(\mathbf{s} \mid T) = \frac{e^{-H(\mathbf{s})/T}}{Z}$$

observable	measures
$e = H/N$	energy per spin: satisfied and broken bonds (local structure)
$m = \frac{1}{N}\sum_i s_i$, $m_{\text{str}} = \left(m^2_{(\pi,0)} + m^2_{(0,\pi)}\right)^{1/2}$	ferromagnetic order, stripe order (J_1–J_2, $g > 1/2$)
$c = N\,\mathrm{Var}(e)/T^2$, $\chi = N\,\mathrm{Var}(\lvert m \rvert)/T$	energy and magnetisation fluctuations; peak at a transition
$U_4 = 1 - \langle m^4 \rangle / 3\langle m^2 \rangle^2$	shape of $P(m)$: $2/3$ ordered, $\to 0$ disordered
$G(r) = \langle s_{\mathbf{x}} s_{\mathbf{x}+r\hat{\mathbf{e}}} \rangle$	how far spin correlations extend
$P(e)$, $P(m)$	full distributions, not only averages

2D Ising: $T_c = 2/\ln(1+\sqrt{2}) = 2.269$ (L. Onsager, Phys. Rev. **65**, 117 (1944); C. N. Yang, Phys. Rev. **85**, 808 (1952)). A generator of spin configurations has to reproduce $p(\mathbf{s} \mid T)$, not only its look.

The cVAE for Spins: a Product of Bernoulli Factors

condition $c = T$ (Ising) or $c = (g, T)$ (J_1–J_2); bits $b_i = (s_i + 1)/2$; one logit $\ell_i(\mathbf{z}, c)$ per site

$$\log p_\theta(\mathbf{s} \,|\, c) \;\geq\; \mathbb{E}_{q_\phi(\mathbf{z}|\mathbf{s},c)}\Big[\sum_{i=1}^{N} b_i \log\sigma(\ell_i) + (1-b_i)\log\big(1-\sigma(\ell_i)\big)\Big] - \mathrm{KL}\big(q_\phi(\mathbf{z} \,|\, \mathbf{s}, c) \,\big\|\, p(\mathbf{z} \,|\, c)\big)$$

prior $p(\mathbf{z}|c) = \mathcal{N}(0, I)$ as in the MNIST cVAE, or learned (below)

Given $\mathbf{z}$, the spins are independent

$p_\theta(\mathbf{s} \,|\, \mathbf{z}, c) = \prod_i p(s_i \,|\, \mathbf{z}, c)$: a mean-field-like product, $\langle s_i \rangle_{\mathbf{z}} = \tanh(\ell_i/2)$,

$$\mathrm{Cov}(s_i, s_j) = \mathrm{Cov}_{\mathbf{z}}\big(\tanh\tfrac{\ell_i}{2}, \tanh\tfrac{\ell_j}{2}\big), \quad i \neq j$$

all correlations must pass through the $d = 16$ latent variables

Two ways to generate

$\mathbf{z} \sim p(\mathbf{z} \,|\, c)$, then

- **Bernoulli**: $s_i = +1$ with probability $\sigma(\ell_i)$
- **most likely**: $\hat{s}_i = \operatorname{sign} \ell_i$

H is a sum of neighbour products $s_i s_j$: independent draws break bonds at random. e, c, $G(1)$ are the sharp tests.

D. P. Kingma, M. Welling, arXiv:1312.6114; K. Sohn, H. Lee, X. Yan, NeurIPS 2015, NeurIPS proceedings

Two Kinds of Tests, Each with a Real-vs-Real Baseline

2D Ising, $L = 30$, 37 temperatures $T \in [1.5, 3.5]$, 256 Swendsen–Wang snapshots per T, split into **training** (128), **held-out** A (64), **held-out** B (64); synthetic: 64 snapshots per T

1. Networks trained on Monte Carlo data

CNN regressor $\hat{T}(\mathbf{s})$, loss $(\hat{T} - T)^2$;
CNN phase classifier trained at $T \leq 1.8$ and $T \geq 2.8$, confidence $\max(p_0, p_1)$

2. The Hamiltonian

$\langle e \rangle$, $\langle |m| \rangle$, c, χ, U_4, $G(r)$ and the distributions $P(e)$, $P(m)$, compared by
$W_1 = \frac{1}{n} \sum_k |x_{(k)} - y_{(k)}|$

3. The generator itself

$-$ELBO on training and held-out data; active latent units ($\mathrm{KL}_j > 0.01$); classifier two-sample test (C2ST): accuracy of a CNN separating real from synthetic, 0.5 = indistinguishable; memorisation: $d_{\mathrm{NN}} = \min_{\mathbf{t} \in \mathrm{train}}(1 - |\mathbf{s} \cdot \mathbf{t}|/N)/2$

Noise units

$R_O = \mathrm{mean}_T |O_{\mathrm{syn}} - O_A| \,/\, \mathrm{mean}_T |O_B - O_A|$;
$R \approx 1$: as close to A as an independent Monte Carlo set

every test runs first on held-out Monte Carlo data, then identically on the synthetic data at the same T C2ST: D. Lopez-Paz, M. Oquab, arXiv:1610.06545

Three cVAEs, Each Adding One Piece of Physics

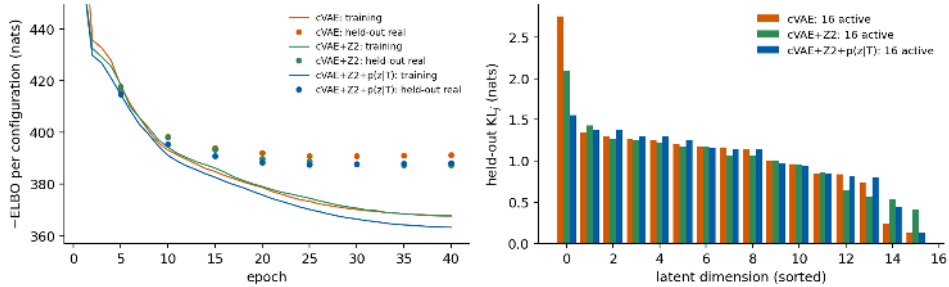
dense encoder $N{+}1 \to 512 \to 256 \to (\boldsymbol{\mu}, \log \boldsymbol{\sigma}^2)$, latent dimension $d = 16$; dense decoder $d{+}1 \to 256 \to 512 \to N$ logits; 40 epochs, Adam

- **cVAE**: prior $\mathcal{N}(0, I)$, raw data (the MNIST model)
- **cVAE+Z2**: the symmetry $H(-\mathbf{s}) = H(\mathbf{s})$ imposed by construction: train on the $m \geq 0$ representative of every snapshot, flip each generated snapshot with probability $\frac{1}{2}$
- **cVAE+Z2+p(z|T)**: conditional prior $p_\psi(\mathbf{z} \mid T) = \mathcal{N}(\boldsymbol{\mu}_\psi(T), \operatorname{diag} \boldsymbol{\sigma}^2_\psi(T))$ from a small network $T \to 64 \to (\boldsymbol{\mu}_\psi, \log \boldsymbol{\sigma}^2_\psi)$: a snapshot reveals its T, so the encoded snapshots of different T occupy different regions of $\mathbf{z}$-space; $\mathbf{z} \sim \mathcal{N}(0, I)$ draws from all of them, $\mathbf{z} \sim p_\psi(\mathbf{z}|T)$ from the region of T

$$\mathrm{KL}(q \,\|\, p_\psi) = \frac{1}{2} \sum_{j=1}^{d} \Big[\log \frac{\sigma^2_{\psi,j}}{\sigma^2_j} + \frac{\sigma^2_j + (\mu_j - \mu_{\psi,j})^2}{\sigma^2_{\psi,j}} - 1 \Big]$$

prior network: K. Sohn, H. Lee, X. Yan, NeurIPS 2015, NeurIPS proceedings

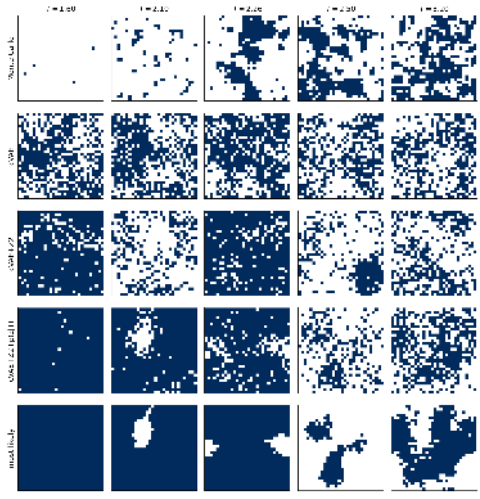
Training Health Does Not Rank the Three Models



left: $-$ELBO per configuration (nats; uniform spins $N \ln 2 = 624$), training and held-out real data; right: held-out KL_j per latent dimension

	cVAE	cVAE+Z2	cVAE+Z2+p(z|T)
$-$ELBO training / held-out	368 / 391	368 / 387	363 / 388
active units ($\mathrm{KL}_j > 0.01$) of 16	16	16	16
spins reproduced from $\mathbf{z} = \boldsymbol{\mu}$	80 %	81 %	81 %

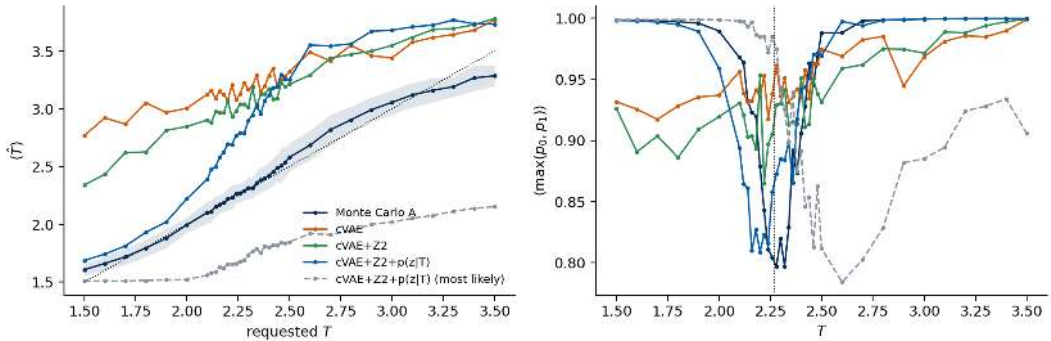
Monte Carlo and Synthetic Ising Configurations



rows: Monte Carlo; cVAE; cVAE+Z2; cVAE+Z2+p(z|T) with Bernoulli sampling; the same $\mathbf{z}$ with $\hat{s}_i = \operatorname{sign} \ell_i$

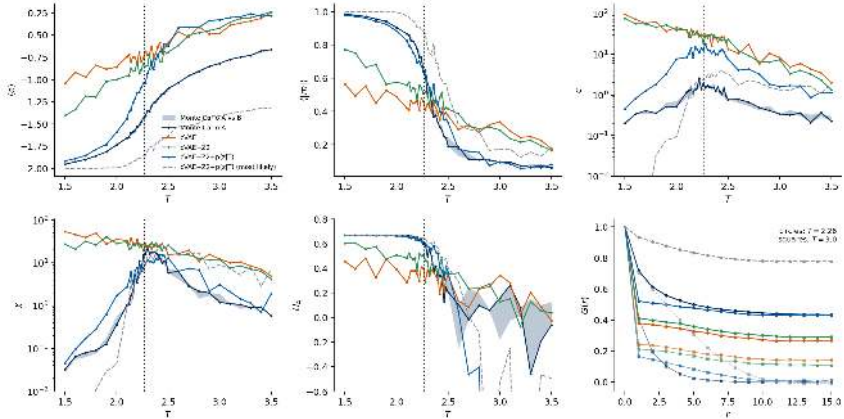
- cVAE and +Z2: the requested T is barely visible
- +p(z|T): ordered below T_c, disordered above; salt-and-pepper noise inside domains
- most likely configuration: no noise, but large smooth domains even at $T = 3.2$

Networks Trained on Monte Carlo Data Judge the Samples



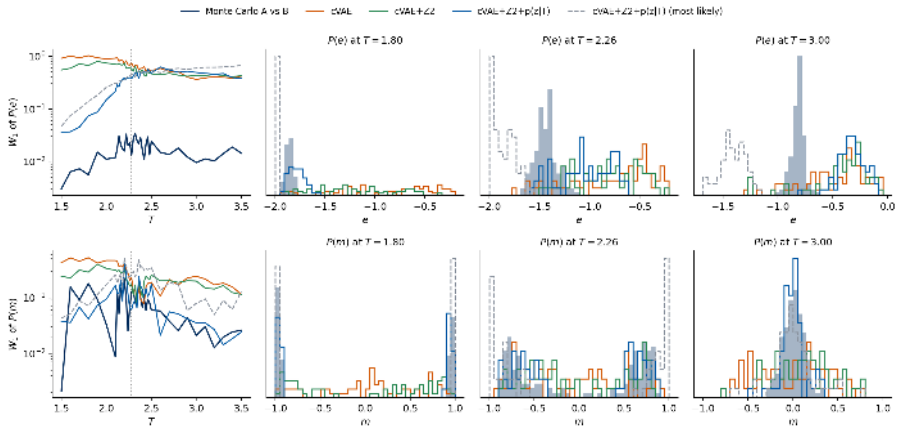
left: $\langle \hat{T} \rangle$ vs requested T (band: $\pm$ std on A); right: $\langle \max(p_0, p_1) \rangle_T$. RMSE of $\hat{T}$: Monte Carlo A 0.12, B 0.12; cVAE 0.98; +Z2 0.84; +p(z|T) 0.62; most likely 0.71. Below T_c only +p(z|T) is read at the requested temperature; above T_c every Bernoulli model looks too hot.

The Order Parameter Is Reproduced, the Energy Is Not



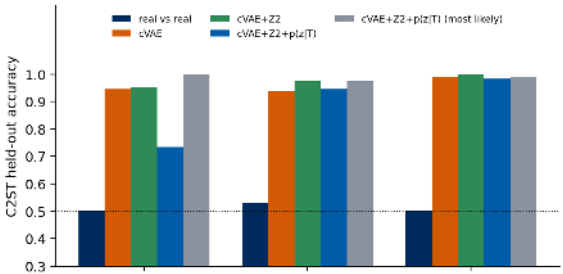
$\langle e \rangle$, $\langle |m| \rangle$, c, χ, U_4 vs T; $G(r)$ at $T = 2.26$ (circles) and $T = 3.0$ (squares); band: Monte Carlo A vs B. Noise ratios of +p(z|T): $R_{|m|} = 2.8$, $R_\chi = 2.0$, $R_{W_1(m)} = 1.3$; $R_e = 34.8$, $R_c = 39.0$, $R_{W_1(e)} = 21.6$. At $T = 2.26$: $\langle e \rangle = -1.43$ (Monte Carlo), -1.04 (Bernoulli), -1.86 (most likely); $G(1) = -\langle e \rangle / 2$: the energy is the nearest-neighbour correlation

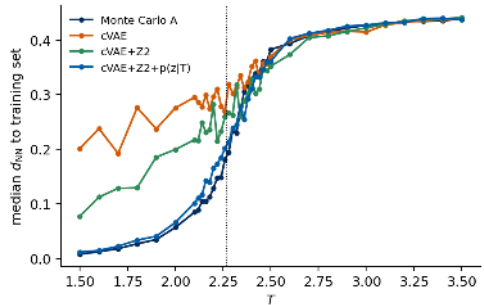
Distributions of Energy and Magnetisation



W_1 between synthetic and Monte Carlo A vs T, against $W_1(A, B)$; $P(e)$, $P(m)$ at $T = 1.8, 2.26, 3.0$. Fraction of snapshots below T_c with $|m| < \langle|m|\rangle_{\mathrm{MC}}/2$: Monte Carlo 1 %, cVAE 45 %, +Z2 27 %, +p(z|T) 1 %. $P(m)$ is right, $P(e)$ is shifted and too broad.

A Classifier Separates Real from Synthetic; Nothing Is Copied

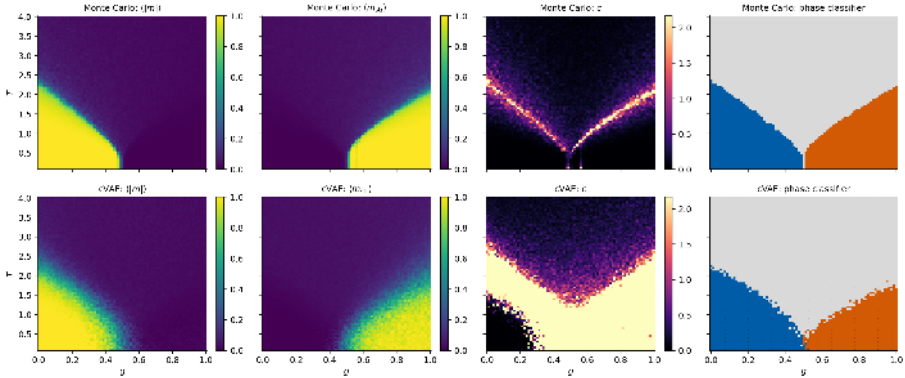




C2ST held-out accuracy at $T = 1.8/2.26/3.0$: real vs real 0.50 / 0.53 / 0.50; +p(z|T) 0.73 / 0.95 / 0.98; most likely 1.00 / 0.98 / 0.99

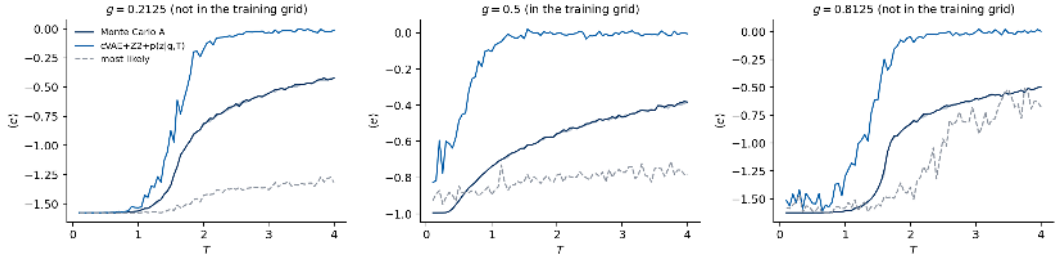
median d_{NN} to the training set at the same T: Monte Carlo 0.03 / 0.18 / 0.43; +p(z|T) 0.03 / 0.20 / 0.43: new configurations, no memorisation

J_1–J_2: the Phase Diagram Is Interpolated



81×79 grid in (g, T), 100 snapshots per point; cVAE+Z2+p(z|g,T) trained on every second g and every second T (dots), generated everywhere; columns: $\langle |m| \rangle$, $\langle m_{\mathrm{str}} \rangle$, c, majority phase of a CNN classifier trained on real data (FM, stripes, PM). Same phase as Monte Carlo A: B 99.7 %; synthetic 95.3 % at seen, 95.4 % at unseen (g, T); most likely configurations 35.4 %.

J_1–J_2: Order Parameters Right, Energy Wrong



R (noise units)	$\langle \lvert m \rvert \rangle$	$\langle m_{\mathrm{str}} \rangle$	$\langle e \rangle$	c
seen (g, T)	4.4	6.4	64.0	2279
unseen (g, T)	4.8	5.9	63.9	1517

−ELBO held-out: 476 (seen), 482 (unseen); stripes alternating along x: 0.50 (Monte Carlo), 0.48 (cVAE). Paramagnet: synthetic $\langle e \rangle \approx 0$, the value for independent spins

What the Generator Reproduced and What It Does Not

Reproduced

- Ising $\langle|m|\rangle$, χ, $P(m)$: $R = 2.8$, 2.0, 1.3; J_1–J_2 $\langle|m|\rangle$, $\langle m_{\mathrm{str}}\rangle$: $R \approx 4.8$, 5.9 (unseen)
- the temperature seen by a regressor below T_c, the phase seen by classifiers trained on real data
- the J_1–J_2 phase diagram at unseen (g, T); both stripe orientations
- new configurations, not copies of the training set

needed: $\mathbb{Z}_2$ symmetry of H by construction, and a conditional prior $p(\mathbf{z}|c)$

Not reproduced

- $\langle e\rangle = H/N$: $R = 34.8$ (Ising), 63.9 (J_1–J_2)
- c, $P(e)$: energy fluctuations an order of magnitude too large
- a C2ST separates real from synthetic at every T

given $\mathbf{z}$ the decoder is a product over sites, a mean-field-like distribution: nearest-neighbour correlations beyond what 16 latent variables carry are lost. Bernoulli sampling breaks bonds, $\operatorname{sign}\ell_i$ over-smooths.

Looking right is not obeying H: synthetic configurations answer questions about the phase; they are not samples of $e^{-H/T}/Z$ and do not replace Monte Carlo for thermodynamics.

Summary

- an **autoencoder** learns a latent space, not a distribution on it: random latent vectors decode to non-digits
- **maximum likelihood** fits p_θ to data, i.e. minimises $\mathrm{KL}(p \,\|\, p_\theta)$
- a **VAE** maximises $\mathrm{ELBO} = \mathbb{E}_q \log p_\theta(\mathbf{x}|\mathbf{z}) - \mathrm{KL}(q \,\|\, p)$, trainable through the reparametrisation trick; the KL term shapes the latent space into $\mathcal{N}(0, I)$: sampling, interpolation and traversals work, single axes need not have a meaning
- a **classifier trained on real data** referees synthetic data: confusion matrices, class distributions, confidence, always against the real-data baseline
- a **conditional VAE** adds a known label $\mathbf{c}$ to encoder and decoder: $\mathbf{c}$ selects what is generated, $\mathbf{z}$ carries the rest
- **spins**: a cVAE conditioned on T or (g, T), with the $\mathbb{Z}_2$ symmetry and a conditional prior, reproduces the phase, the order parameter and the J_1–J_2 phase diagram at unseen (g, T); it misses the energy and its fluctuations: a product of independent spins given $\mathbf{z}$ is not a sample of $e^{-H/T}/Z$

Tutorial: Hands-on

Hands-on (Lab 024)

- train the $d = 16$ VAE without the KL term and sample from $\mathcal{N}(0, I)$; then multiply the KL term by $\beta = 4$: how many active coordinates remain, and how sharp are the samples?
- give the cVAE a mixed condition $\mathbf{c} = \frac{1}{2}(\mathbf{e}_4 + \mathbf{e}_9)$: what does it draw, and what does the classifier say?
- replace the one-hot condition by the single number $y/9$ and measure the confusion matrix again

Thank You!

Questions, project ideas, feedback: marcin.plodzien@uj.edu.pl

Website · GitHub · Google Scholar

